SENIOR 3D WEB ENGINEER · INFRACLOUD3D

Mike Smahin

Senior Frontend / 3D Engineer · Barcelona

15+ years shipping frontend and real-time 3D for the web and games

TypeScript · Angular · React · Three.js · WebGL · WebGPU · WGSL · TSL · Unity / C#

iodiot.github.io · github.com/iodiot · @iodiot · iodiot@gmail.com

01 · WHO I AM

I make demanding real-time 3D run smoothly in the browser, and ship it as product.

ENEMINDS · 2016

Co-founder, lead developer

Redungeon (Unity / C#), published by Nitrome: Apple Editors' Choice, 1M+ downloads per platform.

VIMEO · 2022–2025

3D lead, Vimeo Venues

Crowd engine for virtual events: up to 10,000 avatars at a stable 60+ FPS on everyday laptops.

VIMEO · LIVE EVENTS

Frontend at scale

React / TypeScript features on a large production platform; UI performance, shared design system, testing.

REDCORE · 2025–2026

Product owner & lead engineer

RedCore Town: stabilised in production, scene loading cut 3x, standalone Angular app shipped as an npm package.

02 · FIT

What InfraCloud3D needs, what I bring

YOU NEED	STATUS	EVIDENCE
Large, complex 3D scenes at interactive frame rates	SHIPPED	Vimeo Venues: up to 10,000 avatars at 60+ FPS in the browser · RedCore Town: scene loading cut 3x (GLB, memory, S3 delivery)
WebGPU pipelines & compute	BUILT	webgpu-crowd: WGSL compute, spatial hash grid, 1k to 100k agents · webgpu-ocean: TSL compute FFT
Angular + TypeScript frontend	SHIPPED	Standalone Angular app shipped as an npm package (RedCore) · React / TS at Vimeo scale
Data-driven / geospatial 3D	BUILT	Globe Dashboard: live USGS feed, instanced markers, synced virtualised UI
Guiding other developers	PROVEN	“Three.js from Scratch”, 6 parts incl. WebGPU & TSL and post-processing
LuciadRIA · point clouds · Gaussian Splatting	TRANSFER	Same GPU fundamentals: buffers, instancing, LOD, streaming, compute. Ramp-up plan on slide 11

03 · BIG SCENES IN PRODUCTION

Frame budgets I've held for real users

VIMEO · VENUES

10,000

avatars at a stable 60+ FPS

A virtual conference world where thousands of attendees walk, gather and network as live avatars, right in the browser on everyday laptops. I built the crowd engine from the ground up and owned simulation, rendering and product integration in a team of 10.

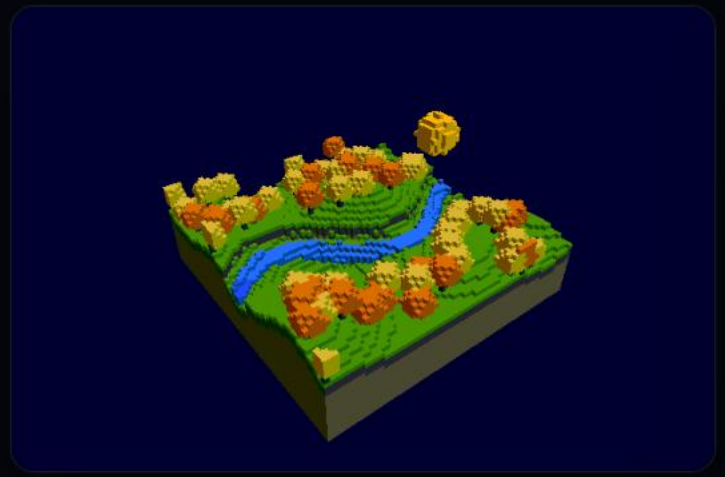
REDCORE · REDCORE TOWN

3x

faster scene loading

Product owner and lead engineer, from development to production. Revived the existing 3D web app and made it stable, optimised GLB assets, memory and S3 delivery. Then built RedCore Town 2100: a huge interactive 3D city with mini-games.

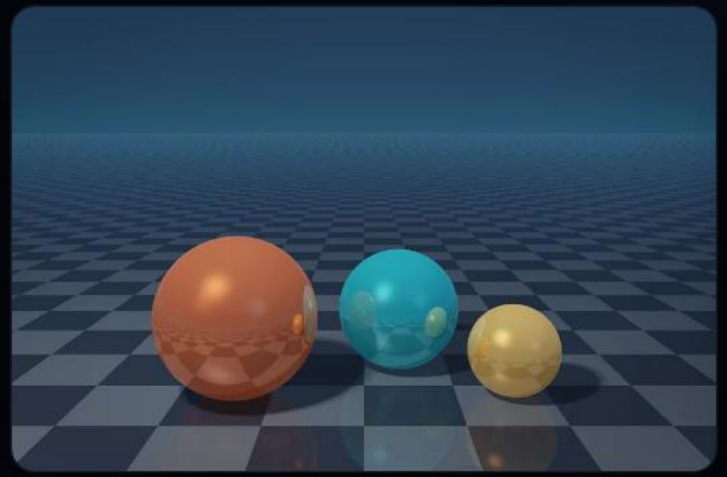
No engine, just shaders: voxels, rays, fluids



GpuVoxels

Ray casting a 64×64×64 voxel grid stored as an OpenGL 3D texture; every pixel walks the grid block by block until it hits.

github.com/iodiot/GpuVoxels



GpuRayTracer

Live ray tracing in a single shader: two light bounces between spheres, soft shadows from an area light.

github.com/iodiot/GpuRayTracer



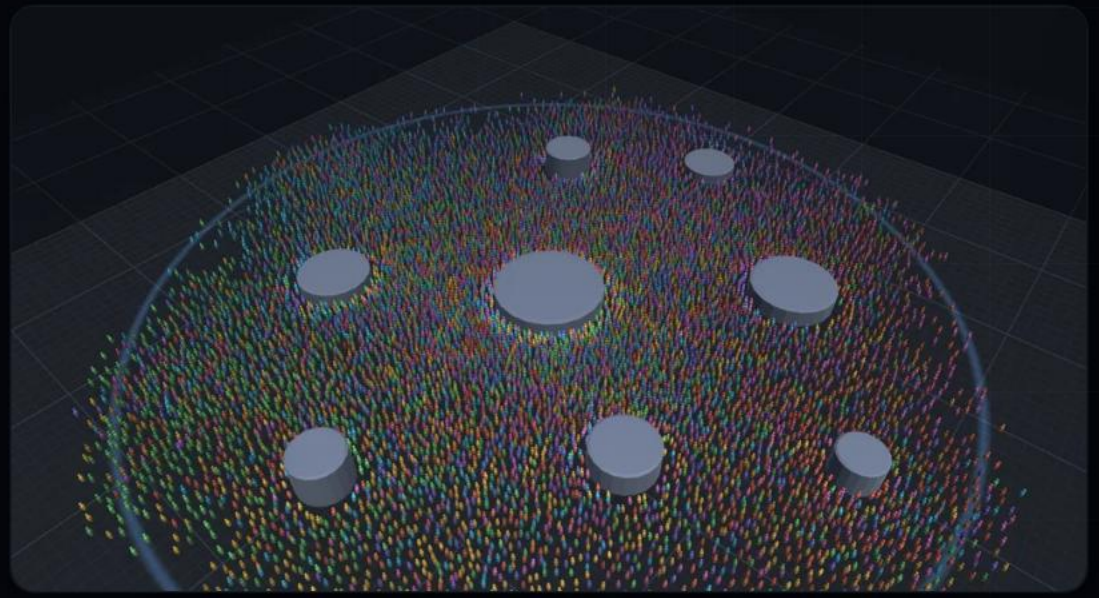
GpuFluid

Navier–Stokes on the GPU: advection, buoyancy, Jacobi pressure solve as shader passes; vorticity confinement, 512 grid.

github.com/iodiot/GpuFluid

C++ · OpenGL · GLSL · 2014–2015 · refreshed 2026 · CMake + GLFW

Compute shaders doing the heavy lifting



webgpu-crowd · 1k → 100k agents

Boids flocking entirely in WGSL compute shaders, with a spatial hash grid for neighbour lookups. Real time, live controls.

`iodiot.github.io/webgpu-crowd`



webgpu-ocean · TSL compute FFT

Every wave simulated on the GPU, from 250 m swells to centimetre ripples; breaking foam, sky reflection, subsurface glow.

`iodiot.github.io/webgpu-ocean`

Globe Dashboard

A live earthquake dashboard around a 3D globe, streaming the USGS real-time feed.

- Every event is one instanced marker; recent events pulse in a shader
- Hover or click to fly the camera to an event
- Virtualised list, filters, KPI cards and SVG charts stay in sync with the globe
- Timeline playback, shareable URL state, keyboard navigation, snapshot fallback when the feed is down

React · TypeScript · Three.js / R3F · zustand · Vitest

iodiot.github.io/globe-dashboard



07 · TEACHING WEBGPU

“Three.js from Scratch”: a free series for frontend devs

Six parts so far, from a first lit cube to shaders, WebGPU and post-processing. Diagrams, real screenshots and a live demo in every part.



Part 5 · WebGPU and TSL

Switch to WebGPURenderer, rebuild materials as TSL node graphs, read the WGSL it generates, then 40,000 particles simulated entirely on the GPU. WebGL 2 fallback.



Part 6 · Post-Processing and Bloom

Render targets and ping-pong by hand, EffectComposer, a custom lens shader, then the same chain on WebGPU with TSL’s RenderPipeline, and selective bloom.

iodiot.github.io/3d-tutorial

The app around the 3D: Angular + TypeScript

REDCORE · 2025–2026

- Owned RedCore Town end to end, from development to production
- Built a standalone **Angular app shipped as an npm package**
- Scene loading cut 3x: GLB assets, memory, S3 delivery
- Created RedCore Town 2100 with a fully redesigned visual identity

VIMEO · 2022–2025

- Live Events features on a large-scale production platform (React / TypeScript)
- Focus on UI performance; contributed to the shared design system
- Better automated testing: fewer regressions, more frequent releases
- 3D lead for Venues inside a team of 10

09 · HONEST GAPS, FAST RAMP-UP

New to me, built on what I already do

LuciadRIA

Another engine on the same fundamentals: scene graph, layers, camera, GPU buffers. I'd learn it through the API docs, your codebase and a small spike on real data.

Point clouds & meshes

Huge instanced draws, spatial hashing, GPU memory and asset-delivery budgets are what I've shipped. Point-cloud LOD and streaming is the same discipline at a larger scale.

Gaussian Splatting

Sorting and blending millions of splats is a GPU compute and rendering problem: the WebGPU compute and custom-shader work I do today.

ALSO ON MY RADAR — Large coordinate spaces: float precision, relative-to-eye / floating origin, depth precision on city- and bridge-scale scenes.

How I'd approach the first 90 days

DAYS 1–30

Learn & measure

- LuciadRIA deep dive on your codebase
- Profile real datasets on target hardware
- Set performance budgets: frame time, GPU memory, load time

DAYS 31–60

Fix the biggest bottleneck

- LOD and streaming for meshes and point clouds
- Culling, instancing, worker offloading
- Automated perf checks so wins don't regress

DAYS 61–90

WebGPU migration path

- Feature detection with a WebGL fallback
- First compute workload moved to the GPU
- Spike: Gaussian Splatting on a real survey

11 · QUESTIONS FOR YOU

What I'd like to understand

1. Where does LuciadRIA hit its limits today, and where would WebGPU come in?
2. Typical dataset size (points, triangles, GB) and target hardware?
3. Team size, and how 3D work is split between frontend and backend?

THANK YOU

Let’s build it on the GPU.

Mike Smahin · Senior Frontend / 3D Engineer · Barcelona

Email	iodiot@gmail.com
Portfolio	iodiot.github.io
GitHub	github.com/iodiot
X	@iodiot
Tutorial	iodiot.github.io/3d-tutorial
Crowd	iodiot.github.io/webgpu-crowd
Ocean	iodiot.github.io/webgpu-ocean
Globe	iodiot.github.io/globe-dashboard